

Plan Completo para Construir un Sistema de Trading Multi-ETF

El objetivo es construir un sistema de trading capaz de gestionar una **cartera diversificada de ETFs** en renta variable y renta fija con estándares profesionales. Este documento guía cada fase del proyecto, desde la selección del universo y adquisición de datos hasta la arquitectura de software, y proporciona enlaces a fuentes clave para profundizar.

1. Selección del Universo

Un universo adecuado reduce sesgos y facilita la gestión. Utilizaremos ETFs líquidos que representen clases de activos globales:

- **Renta variable estadounidense:** SPY/IVV (S&P 500), QQQ (Nasdaq), IWM (Russell 2000). Estas selecciones siguen la práctica de estudiar índices representativos y son similares a las empleadas en investigaciones académicas como TrendFolios, que construye señales de tendencia sobre un universo de ETFs representativos ¹.
- **Renta variable global:** EFA (mercados desarrollados) y EEM (emergentes) ².
- **Renta fija gubernamental:** SHY (1-3 años), IEF (7-10 años) y TLT (20 años). TrendFolios también emplea ETFs de bonos para capturar señales de tendencias en duración ¹.
- **Crédito corporativo:** LQD (Investment Grade) y HYG (High Yield) ².
- **Inflación:** TIP (bonos ligados a la inflación), para cubrir riesgos de inflación.

Mantener un universo estático evita el sesgo de supervivencia. Podremos ampliar el universo a sectores o factores específicos (p.ej., XLK, XLF) una vez validado el núcleo.

2. Adquisición y Limpieza de Datos

Para backtesting serio se requiere un **conjunto de datos robusto**: precios ajustados (Close ajustado y dividendos) y, si es posible, *forward-adjusted* para evitar saltos por dividendos; calendario de trading, volumen y eventos corporativos. El artículo de QuantInsti subraya la importancia de la **diversificación y datos fiables** para gestión de carteras y riesgos ³. Asimismo, los frameworks profesionales como btester permiten cargar datos históricos en forma de DataFrames y simular posiciones y trades ⁴.

Se recomienda:

- **Fuentes primarias de datos:** APIs de proveedores como Finazon, TwelveData o AlphaVantage. Una guía de APIs popularizada en 2025 muestra que proveen datos históricos y en tiempo real para acciones y ETFs ⁵.
- **Preprocesado:** eliminar días con precios nulos, ajustar por splits y dividendos, alinear series temporales por calendario e imputar valores faltantes según normas definidas.

3. Estrategias Base y Benchmark

Antes de implementar modelos complejos, es vital construir **estrategias baseline** que sirvan de comparación y garanticen robustez. Estas estrategias deben funcionar con costes realistas y servir como control. Algunas propuestas:

1. **Trend/Time-Series Momentum:** calculada mediante medias móviles o rupturas de máximos/mínimos. TrendFolios utiliza medias móviles de varias ventanas y fusiona señales a través de un voto mayoritario ⁶. Se asigna peso positivo cuando la media de corto plazo supera a la de largo plazo, y negativo en caso contrario. La ventaja de la estrategia de trend multi-activo es documentada en numerosos estudios y su implementación es relativamente simple.
2. **Momentum Cross-Sectional:** ordenar ETFs por rendimiento en ventanas de 6–12 meses y seleccionar los de mayor rendimiento (long) y menor rendimiento (short). La metodología se basa en la literatura de factor investing y se puede implementar con el tutorial de Leapcell para momentum ⁷. ⁸.
3. **Baselines de Inversión Equitativa:** por ejemplo, un índice de igual ponderación o riesgo paritario. El artículo de B2Broker explica que la cartera de *risk parity* asigna igual riesgo a cada clase de activo ⁹, minimizando la dependencia de un único activo. Otra alternativa es el enfoque de **Hierarchical Risk Parity (HRP)** de López de Prado, que usa clustering jerárquico y grafos para mejorar la diversificación ¹⁰.

Implementaremos estas estrategias como benchmarks; cualquier modelo avanzado deberá superar su rendimiento ajustado por riesgos y costes.

4. Ingeniería de Características y Modelado

Aquí se diseñan las variables de entrada (features) y los modelos de predicción.

4.1 Características tradicionales

- **Retornos y volatilidad multi-horizonte:** retornos logarítmicos sobre 1, 5, 20 y 60 días; volatilidad histórica; medidas de asimetría y curtosis.
- **Volumen y liquidez:** ratio volumen/media, bid-ask proxies (cuando se disponga).
- **Indicadores técnicos:** RSI, MACD, medias móviles exponenciales.
- **Factores fundamentales (opcional):** PER, precio-valor contable, etc. Son útiles para estrategias de valor/quality, como se sugiere en el enfoque de tres pilares (equal-weight, high-quality momentum, robust value) ¹¹.

4.2 Grafos y Topología

Para construir señales innovadoras se puede modelar el mercado como un **grafo de correlaciones** donde los nodos son ETFs y los pesos son funciones de la correlación a corto plazo. Aplicar el **Laplaciano del grafo** permite suavizar retornos y detectar desviaciones locales (mispricing). El blog de Jerry A. Smith explica cómo difundir los retornos mediante el operador $\exp(-\tau L)$ para obtener un retorno esperado y usar el residuo como señal de reversión ¹². Esta señal puede ser modulada según el régimen de mercado.

La **homología persistente** mide la topología del grafo: captura la aparición y desaparición de ciclos o “huecos”. Cuando el mercado se fragmenta en muchas componentes desconectadas, indica posibles crisis ¹³. Estas características topológicas servirán como **indicadores de régimen**: por ejemplo,

amplificar el peso en bonos durante regímenes de crisis y aumentar riesgo en acciones en regímenes de estabilidad.

4.3 Modelos de Predicción y Señales

Se combinarán distintas familias de modelos:

- **Modelos lineales/regresión:** para predecir retornos o probabilidades de subida. Sencillos y rápidos, útiles como base.
- **Modelos de árbol (Random Forest, Gradient Boosting):** capturan relaciones no lineales entre features. Su interpretabilidad es razonable y se integran bien en pipelines.
- **Redes neuronales y RNN/LSTM:** para capturar patrones temporales complejos; se debe evitar el overfitting y considerar la estabilidad fuera de muestra.
- **Modelos de ensemble:** combinar señales de trend, momentum y grafos usando votación o blending. La idea de fusionar una estrategia de red neuronal con topología se menciona en el vídeo de Jerry Smith, en el que utiliza redes y persistent homology como fuentes complementarias de señales ¹².

Los modelos no deben utilizarse en solitario; la señal final se obtiene combinando múltiples modelos y modulándolos según el régimen.

5. Construcción de Cartera y Optimización

5.1 De Señales a Pesos

Las señales deben convertirse en pesos de portafolio mediante algoritmos de asignación. Opciones comunes:

- **Volatility Targeting:** ajustar la exposición de cada activo para alcanzar una volatilidad objetivo. Esto requiere estimar la volatilidad ex ante y ajustar pesos proporcionalmente.
- **Mean-Variance (Modern Portfolio Theory – MPT):** optimiza la combinación para maximizar retorno esperado a un nivel dado de riesgo. Debe incluir restricciones (suma de pesos=1, pesos ≥ 0 o permitir cortos). La explicación de MPT enfatiza que el objetivo es maximizar la rentabilidad para un riesgo dado y que la varianza de la cartera se calcula mediante la covarianza de los activos ¹⁴.
- **Minimum Variance/Maximum Sharpe:** son formulaciones específicas de MPT; se resuelven mediante programación convexa o heurísticas ¹⁵.
- **Risk Parity y HRP:** asignan riesgos de manera equilibrada. El artículo de B2Broker señala que el risk parity busca que cada activo aporte la misma cantidad de riesgo ⁹, mientras que HRP usa clustering jerárquico y no requiere invertir la matriz de covarianza ¹⁰.

5.2 Gestión de Exposición y Turnover

Se deben imponer límites a la asignación por activo (p.ej., 10 %-30 % para cada ETF), controlar el apalancamiento global y monitorizar el **turnover** para minimizar costes. Las plataformas de backtesting como Backtrader ofrecen soporte para aplicar costes de comisión, slippage, y trabajar con datos de múltiples timeframes ¹⁶.

6. Gestión de Riesgo

Un modelo serio incorpora reglas de riesgo desde el principio:

1. **Medición de riesgo:** volatilidad, VaR/ES (Value at Risk/Expected Shortfall), y drawdowns. Las fuentes de QuantStart subrayan la importancia de calcular métricas de rendimiento, drawdown, ratios Sharpe/Sortino, y visualizar la evolución de la cartera ¹⁷.
2. **Límites y stops:** asignar stops lógicos (basados en volatilidad o señales) y límites de pérdida diaria o mensual.
3. **Gestión de colas:** diversificar regímenes (trend, mean-reversion, carry) para no depender de un único entorno; mantener exposición a bonos y cash para amortiguar crisis.

7. Backtesting y Validación

Realizar un backtest riguroso es esencial para evitar **sesgos y sobreajuste**. Debe incluir:

- **Arquitectura event-driven:** frameworks como bttester o Backtrader permiten simular trades a nivel de órdenes y eventos ⁴ ¹⁶.
- **Walk-forward y cross-validation temporal:** dividir el historial en ventanas de entrenamiento y validación, avanzando en el tiempo, para evaluar la robustez fuera de muestra.
- **Purgar/embargar:** eliminar datos cercanos al punto de prueba para evitar *look-ahead bias* y *leakage*.
- **Costes realistas:** incorporar comisiones por operación y slippage. QuantStart explica que un framework de backtesting serio debe incluir manejo de costes, optimización y evaluación de métricas ¹⁸.
- **Análisis de sensibilidad y robustez:** variar parámetros (por ejemplo, ventanas de medias móviles) para asegurarse de que el rendimiento no depende de un único set de valores (combatir el *data snooping*).

8. Arquitectura de Software

Diseñaremos una arquitectura modular y extensible. Un ejemplo de módulos basados en prácticas de frameworks open-source:

1. **DataLoader:** descarga y prepara los datos de los ETFs, gestiona splits/dividendos. Basado en los ejemplos de bttester ⁴.
2. **SignalModel:** calcula señales de cada estrategia (trend, momentum cross-sectional, mean-reversion en grafo, régimen topológico). Permite registrar nuevos modelos fácilmente.
3. **PortfolioBuilder:** combina señales y genera pesos respetando restricciones de riesgo y límites de exposición.
4. **RiskManager:** aplica control de riesgo (volatility targeting, drawdown stops, VaR) y comunica al portafolio si debe reducir exposición.
5. **Backtester:** ejecuta la simulación event-driven y genera reportes de métricas y transacciones; puede implementarse con Backtrader, PyAlgoTrade o QSTrader ¹⁹.
6. **Execution/TradeService:** interfaz con el broker simulado o real, gestiona órdenes (limit, market). La guía de Biz4Group para construir plataformas de trading recomienda un backend robusto con APIs, seguridad y pruebas exhaustivas ²⁰.
7. **Monitor de Riesgo y Panel:** visualiza en tiempo real la exposición, drawdown y señales; integra alertas. Las soluciones AI en gestión de ETFs se apoyan en monitorización en tiempo real y rebalanceo automatizado ²¹.

Un diseño limpio facilita la extensibilidad y permite integrar fácilmente nuevas fuentes de datos o modelos.

9. Herramientas de Desarrollo

Se recomienda usar Python y librerías de ciencia de datos (pandas, numpy, scipy) así como paquetes específicos:

- **NetworkX** y **scipy.sparse** para construir y analizar el grafo de correlaciones y aplicar el operador laplaciano.
- **GUDHI** o **Ripser** para calcular homología persistente (Vietoris–Rips complexes) ¹³.
- **Backtrader**, **bt** o **vectorbt** para backtesting; Backtrader soporta multi-timeframe y live trading ¹⁶.
- **PyPortfolioOpt** para implementar MPT, minimum-variance y otras optimizaciones; **cvxpy** para optimización convexa.
- **Dash/Streamlit** para construir paneles interactivos y monitoreo en tiempo real.

10. Roadmap de Desarrollo

1. **Preparación de datos (1–2 semanas):** obtener históricos de ETFs, ajustar y validar la calidad. Implementar `DataLoader`.
2. **Backtester básico (2–3 semanas):** construir un motor que soporte rebalanceo, costes y métricas. Probarlo con una estrategia de tendencia simple para verificar el flujo completo.
3. **Implementar estrategias baseline (3–4 semanas):** trend multi-asset, momentum cross-sectional, risk parity/HRP. Calibrar parámetros y analizar resultados.
4. **Desarrollar grafo y régimen topológico (4–6 semanas):** construir la red de correlaciones y los indicadores de homología; diseñar la señal de mispricing y su integración en el portafolio.
5. **Modelos avanzados y ensembles (variable):** integrar modelos ML y redes; experimentar con ensembling, atención a overfitting.
6. **Sistema de ejecución y monitorización (3–4 semanas):** construir API/interface para paper trading; integrar un panel de control.
7. **Documentación y CV:** documentar cada módulo con notebooks y comentarios; describir en tu CV la arquitectura, pruebas, resultados de backtest y qué aprendiste (por ejemplo, mencionar que aplicaste HRP para diversificación y combinaste señales de topología y ML). La guía de QuantInsti enfatiza que la diversificación, gestión de riesgo y monitoreo son claves para la gestión de portafolios ²².

11. Lecturas Recomendadas y Recursos Adicionales

- **Papers y Artículos académicos**
- **TrendFolios (2023)** – Explica la construcción de señales de tendencia/contratendencia en una cartera de ETFs y ofrece un universo implementable ¹ ⁶.
- **“Hierarchical Risk Parity” de López de Prado (2016)** – Introduce HRP y justifica su ventaja sobre la optimización clásica ¹⁰.
- **“The Market Has a Shape” de Jerry A. Smith (2022)** – Describe el uso de laplacianos de grafos y homología persistente para detectar mispricing y regímenes de mercado ¹² ¹³.
- **Blogs y guías**
- **QuantStart** – Artículos sobre backtesting, optimización, frameworks y métricas ¹⁷ ¹⁸.
- **Backtrader Complete Guide** – Tutorial para aprender el framework de backtesting en Python ¹⁶.

- **Analytics Vidhya** – Explicación de la teoría moderna de carteras y ejemplos de optimización ¹⁴ .
- **B2Broker y otros blogs sobre risk parity** – Para comprender estrategias de riesgo equilibrado ⁹ .
- **Repositorios GitHub**
- **bttester** (lenguaje Python) – Framework open-source para backtesting multi-activos, que permite definir estrategias derivando una clase `Strategy` y gestionar posiciones y trades ⁴ .
- Otros repositorios de estrategias momentum y HRP disponibles públicamente; sugerencia: buscar “multi-asset momentum backtest” y “HRP portfolio” en GitHub para código base.

12. Conclusión

Construir un algoritmo de trading competitivo no es solo implementar un modelo predictivo; requiere integrar **universo apropiado, gestión de datos, estrategias baseline robustas, optimización de portafolios, gestión de riesgo y backtesting riguroso**. Este plan proporciona un mapa detallado para desarrollar un sistema multi-ETF sólido mientras aprendes ingeniería financiera, matemáticas aplicadas y programación. Cada fase incorpora lecturas recomendadas y mejores prácticas para evitar errores comunes como el *data snooping* y la sobreoptimización. Con disciplina y documentación, el proyecto no solo servirá como plataforma de trading, sino también como evidencia valiosa en tu CV del dominio adquirido en finanzas cuantitativas y desarrollo de software.

¹ ² ⁶ TrendFolios®: A Portfolio Construction Framework for Utilizing Momentum and Trend-Following In a Multi-Asset Portfolio

<https://arxiv.org/html/2506.09330v1>

³ ²² Portfolio Management Of Multiple Strategies Using Python

<https://blog.quantinsti.com/portfolio-management-strategy-python/>

⁴ GitHub - pawelkn/bttester: Python framework optimized for running backtests on multiple asset portfolios

<https://github.com/pawelkn/bttester>

⁵ ²¹ AI-Powered ETF Management. Portfolio Strategies with Machine... | by Leo Mercanti | Medium

<https://leomercanti.medium.com/ai-powered-etf-management-8233c346daeb>

⁷ ⁸ How to Integrate Momentum Strategy in Python: A Step-by-Step Guide | Leapcell

<https://leapcell.io/blog/momentum-strategy-in-python>

⁹ Risk Parity Strategy - An Investor's Safe Choice.

<https://b2broker.com/news/risk-parity-strategy-an-investors-safe-choice/>

¹⁰ Hierarchical Risk Parity - Wikipedia

https://en.wikipedia.org/wiki/Hierarchical_Risk_Parity

¹¹ Algorithmic Trading: Building a Three-Pillar Quantitative Strategy | by Saurav | Nov, 2025 | Medium

<https://medium.com/@writeronepagecode/algorithmic-trading-building-a-three-pillar-quantitative-strategy-c07f9a41d360>

¹² ¹³ The Market Has a Shape. And It's Broken. | by Dr. Jerry A. Smith | Dec, 2025 | Medium

<https://medium.com/@jsmith0475/the-market-has-a-shape-and-its-broken-bcf101d33e6a>

¹⁴ ¹⁵ Portfolio Optimization in Python | Modern Portfolio Theory in Python

<https://www.analyticsvidhya.com/blog/2021/04/portfolio-optimization-using-mpt-in-python/>

¹⁶ Backtrader for Backtesting (Python) - A Complete Guide - AlgoTrading101 Blog

<https://algotrading101.com/learn/backtrader-for-backtesting/>

17 18 19 **Backtesting Systematic Trading Strategies in Python: Considerations and Open Source Frameworks | QuantStart**

<https://www.quantstart.com/articles/backtesting-systematic-trading-strategies-in-python-considerations-and-open-source-frameworks/>

20 **Multi Asset AI Trading Platform Development: Step-by-Step Guide**

<https://www.biz4group.com/blog/multi-asset-trading-platform-development-guide>